Problem Set 4

Due February 13, 2026 at 11:59 PM Pacific

Turn in your problem set by Gradescope. Include any code you write in your solutions as well. Mark down how much time you spent on the homework set.

1. [10 points] **Information Filter:** In this problem you will derive an equivalent form of the Kalman filter called the information filter, which is advantageous for some problems. Consider the quantities $\Omega_{t|t} := \Sigma_{t|t}^{-1}$, $\Omega_{t|t-1} := \Sigma_{t|t-1}^{-1}$, called the information matrix, and $\xi_{t|t} := \Sigma_{t|t}^{-1}\mu_{t|t}$, $\xi_{t|t-1} := \Sigma_{t|t-1}^{-1}\mu_{t|t-1}$ called the information state.

   (a) Derive recursive expressions for $\Omega_{t|t}$ and $\Omega_{t|t-1}$ from the Kalman filter equations. Your expressions should each require, at most, a single matrix inversion (aside from R_t^{-1} and Q_t^{-1}, which can be precomputed). The following matrix inversion identity will be useful: $(A+UCV)^{-1} = A^{-1} - A^{-1}U(VA^{-1}U + C^{-1})^{-1}VA^{-1}$ for matrices A, U, C, and V.

   Solution:

   (1) **prediction step**

   Take inverse on both sides of Kalman Filter equation

   $$(\Sigma_{t+1|t})^{-1} = (A_t\Sigma_{t|t}A_t^T + Q_t)^{-1}$$

   Use Woodbury matrix identity $(A+UCV)^{-1} = A^{-1} - A^{-1}U(VA^{-1}U + C^{-1})^{-1}VA^{-1}$.
   Set $Q_t = A, A_t = U, \Sigma_{t|t} = C, A_t^T = V$, we can get:

   $$\boxed{\Omega_{t+1|t} = Q_t^{-1} - Q_t^{-1}A_t(A_t^TQ_t^{-1}A_t + \Omega_{t|t})^{-1}A_t^TQ_t^{-1}}$$

   (2) **update step**

   $$(\Sigma_{t|t})^{-1} = [\Sigma_{t|t-1} - \Sigma_{t|t-1}C_t^T(C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1}C_t\Sigma_{t|t-1}]^{-1}$$

   Use Woodbury matrix identity again and set $\Sigma_{t|t-1} = A, -\Sigma_{t|t-1}C_t^T = U, (C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1} = C, C_t\Sigma_{t|t-1} = V$, we can get:

   $$\Omega_{t|t} = \Omega_{t|t-1} + \Omega_{t|t-1}\Sigma_{t|t-1}C_t^T(-C_t\Sigma_{t|t-1}\Omega t|t-1\Sigma_{t|t-1}C_t^T + C^{-1})^{-1}C_t$$

   since $\Omega_{t|t-1}\Sigma_{t|t-1} = I$

   $$\boxed{\Omega_{t|t} = \Omega_{t|t-1} + C_t^TR_t^{-1}C_t}$$

   (b) Similarly, derive recursive expressions for $\xi_{t|t}$ and $\xi_{t|t-1}$ from the Kalman filter equations. Again, your expressions should require no more than a single matrix inverse except for R_t^{-1} and Q_t^{-1}. The following matrix identity will be useful: $(A+UCV)^{-1}UC = A^{-1}U(C^{-1} + VA^{-1}U)^{-1}$.

   Solution:

   (1) **prediction step**

   Start from prediction step in Kalman Filter:

   $$\mu_{t+1|t} = A_t\mu_{t|t} + B_tu_t$$

   Substitute μ with information matrix and information state:

   $$\Omega_{t+t|t}^{-1}\xi_{t+t|t} = A_t\Omega_{t|t}^{-1}\xi_{t|t} + B_tu_t$$

   $$\boxed{\xi_{t+1|t} = \Omega_{t+1|t}(A_t\Omega_{t|t}^{-1}\xi_{t|t} + B_tu_t)}$$

(2) **update step**
Start from Kalman Filter equation:

$$\begin{aligned}
\mu_{t|t} =& \mu_{t|t-1} + \Sigma_{t|t-1}C_t^T(C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1}(y_t - C_t\mu_{t|t-1}) \\
\Sigma_{t|t}\xi_{t|t} =& \Sigma_{t|t-1}\xi_{t|t-1} + \Sigma_{t|t-1}C_t^T(C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1}(y_t - C_t\mu_{t|t-1}) \\
\xi_{t|t} =& \Omega_{t|t}\Sigma_{t|t-1}\xi_{t|t-1} + \Omega_{t|t}\Sigma_{t|t-1}C_t^T(C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1}y_t \\
& - \Omega_{t|t}\Sigma_{t|t-1}C_t^T(C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1}C_t\Sigma_{t|t-1}\xi_{t|t-1} \\
=& \Omega_{t|t}\Sigma_{t|t-1}\xi_{t|t-1} + \Omega_{t|t}\Sigma_{t|t-1}C_t^T(C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1}y_t \\
& - \Omega_{t|t}(\Sigma_{t|t-1} - \Sigma_{t|t})\xi_{t|t-1} \\
=& \xi_{t|t-1} + \Omega_{t|t}\Sigma_{t|t-1}C_t^T(C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1}y_t \\
=& \xi_{t|t-1} + \Omega_{t|t}\Omega_{t|t-1}^{-1}C_t^T(C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1}y_t
\end{aligned}$$

Apply matrix identity on the term $\Omega_{t|t-1}^{-1}C_t^T(C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1}$ with $A = \Omega_{t|t-1}, U = C_t^T, V = C_t, C^{-1} = R_t$ and substitute $\Omega_{t|t}$ with $\Omega_{t|t-1} + C_t^T R_t^{-1} C_t$ (from update step equation)

$$\begin{aligned}
\xi_{t|t} =& \xi_{t|t-1} + (\Omega_{t|t-1} + C_t^T R_t^{-1} C_t)(\Omega_{t|t-1} + C_t^T R_t^{-1} C_t)^{-1} C_t^T R_t^{-1} y_t \\
=& \xi_{t|t-1} + C_t^T R_t^{-1} y_t
\end{aligned}$$

Thus, we have:

$$\boxed{\xi_{t|t} = \xi_{t|t-1} + C_t^T R_t^{-1} y_t}$$

(c) Matrix inversion dominates the computational complexity of the Kalman filter equations (with larger matrices taking longer to invert). With this in mind, when might one prefer the standard form or the information form of the Kalman filter?

Solution:

The information filter is functionally equivalent to the Kalman filter, but they have different computational complexity.
In the update phase, the computation of the Kalman gain $\Sigma_{t|t-1}C_t^T(C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1}$ requires matrix inverse which is usually $O(n^3)$ complexity. The matrix is the same dimension as measurement vector which can be high dimensional for example in multi-sensor data fusion problem. In the informaion filter, the prediction step requires matrix inverse which is the same dimension as state vector. Thus, in general, if the number of measurement variables is larger than number of state variables, we prefer information filter and vice versa.
Other than computational complexity, information filters have the advantage that information matrices can be initialized to be 0 which means the initial states are completely unknown. Whereas in Kalman filters, a finite covariance matrix must be given.

2. [10 points] **Linearizing Dynamics:** Consider the rotational motion of a satellite, which can be modeled by Euler's equations of rigid body motion,

$$\begin{aligned} J_x\dot{\omega}_x &= (J_y - J_z)\omega_y\omega_z + \tau_x \\ J_y\dot{\omega}_y &= (J_z - J_x)\omega_z\omega_x + \tau_y \\ J_z\dot{\omega}_z &= (J_x - J_y)\omega_x\omega_y + \tau_z, \end{aligned}$$

where $[\omega_x \quad \omega_y \quad \omega_z]^T$ is the angular velocity vector, $J = \text{diag}([J_x \quad J_y \quad J_z])$ is the inertia tensor in principal axes, and $\tau = [\tau_x \quad \tau_y \quad \tau_z]^T$ is the input torque from thrusters.

(a) Use a first order discrete time approximation to find a difference equation model for the satellite. Use the time step $\delta t = 0.001$s.

Solution: First order discrete time approximation for the first equation:

$$\begin{aligned} I_x\dot{\omega}_x &= (I_y - I_z)\omega_y\omega_z + \tau_x \\ \frac{I_x}{\delta t}(\omega_x(t+1) - \omega_x(t)) &= (I_y - I_z)\omega_y(t)\omega_z(t) + \tau_x(t) \\ \omega_x(t+1) &= \omega_x(t) + \frac{\delta t}{I_x}[(I_y - I_z)\omega_{y(t)}\omega_z(t) + \tau_x(t)] \end{aligned}$$

Set $x_t = [\omega_x(t), \omega_y(t), \omega_z(t)]^T$, then:

$$x_t = x_{t-1} + \begin{bmatrix} \frac{\delta t}{I_x}[(I_y - I_z)\omega_y(t)\omega_z(t) + \tau_x(t)] \\ \frac{\delta t}{I_y}[(I_z - I_x)\omega_z(t)\omega_x(t) + \tau_y(t)] \\ \frac{\delta t}{I_z}[(I_x - I_y)\omega_x(t)\omega_y(t) + \tau_z(t)] \end{bmatrix}$$

(b) Find the Jacobian of the resulting nonlinear discrete-time dynamics.

Solution:

$$J_d = \frac{\partial x_{t+1}}{\partial x_t} = \begin{bmatrix} 1 & \frac{\delta t}{I_x}(I_y - I_z)\omega_z(t) & \frac{\delta t}{I_z}(I_y - I_z)\omega_y(t) \\ \frac{\delta t}{I_y}(I_z - I_x)\omega_z(t) & 1 & \frac{\delta t}{I_y}(I_z - I_x)\omega_x(t) \\ \frac{\delta t}{I_z}(I_x - I_y)\omega_y(t) & \frac{\delta t}{I_z}(I_x - I_y)\omega_x(t) & 1 \end{bmatrix}$$

(c) Suppose the satellite gets angular velocity measurements from on-board gyroscopes with the measurement equation

$$y_t = \text{sat}(\omega_t) + v_t,$$

where $\text{sat}(\cdot)$ is the element-wise saturation function defined as follows. Let $\omega_t = [\omega_t^1 \quad \omega_t^2 \quad \omega_t^3]^T$. Then $\text{sat}(\cdot)$ returns a vector with ith element ω_t^i if $|\omega_t^i| \leq c$ for some threshold value c, and $\text{sign}(\omega_t^i)c$ if $|\omega_t^i| > c$ is above the threshold (where $\text{sign}(\cdot)$ returns 1 or -1 depending on the sign of its argument). Find the Jacobian of the measurement function.

Solution:

$$J_m = \frac{\partial y_t}{\partial x_t} = \begin{bmatrix} \delta(\omega_x(t)) & 0 & 0 \\ 0 & \delta(\omega_y(t)) & 0 \\ 0 & 0 & \delta(\omega_z(t)) \end{bmatrix}$$

where $\delta(t) = \begin{cases} 1 & |t| \leq c \\ 0 & \text{otherwise} \end{cases}$

3. [5 points] **Extended KF:** Simulate the above nonlinear discrete time system in Python, and implement an extended Kalman filter to estimate its state. Let $J = \text{diag}([1 \quad 5 \quad 5])$, $\omega_0 = [10 \quad 0.1 \quad 0.1]^T$, $c = 10$, the process noise has covariance $Q = 0.004I$, and the measurement noise has covariance $R = 0.1I$. Let your initial state estimate be $\mu_0 = [10 \quad 0 \quad 0]^T$, $\Sigma_0 = \text{diag}([1 \quad 1 \quad 1])$. Assume the input torque is zero. Attach plots showing the satellite motion, and the performance of the filter. **Solution:**

   Several details to pay attention to:
   (1) In the actual simulation part (where you acquire true states and true measurements), use full nonlinear models and add noise to states and measurements.
   (2) In the EKF filter, only use estimation states since filters don't have access to true states. To be specific, use estimation results to calculate A matrices and C matrices.
   (3) Use full nonliear models if possible. I.e. in the EKF predict step, use $f(x, u)$ instead of A; in the update step, use $g(x)$ instead of C.

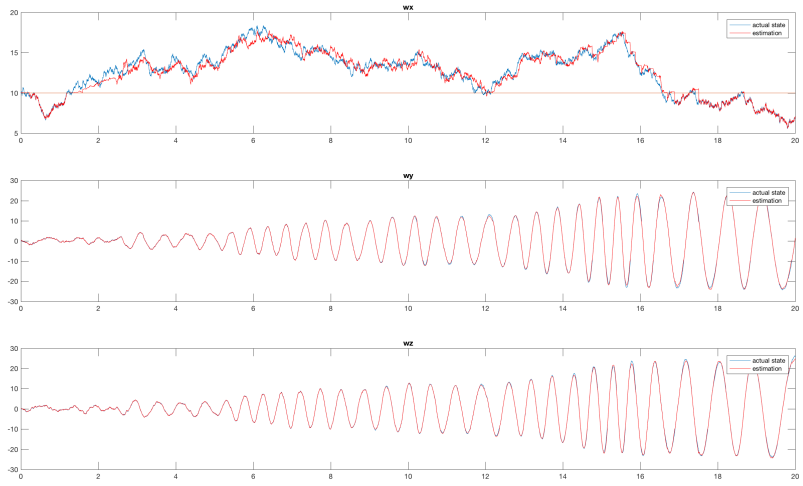


Figure 1: Plots of the satellite motion and the EKF filter. Blue lines are the simulation of satellite motion in three directions. Red lines are filter results. Measurement saturation value c = 10.

4. [30 points] **Nonlinear Filters:** Consider the discrete time nonholonomic mobile robot model we saw in class

$$\begin{aligned} P^x_{t+1} &= P^x_t + \delta t\, s_t \cos\Theta_t + W^x_t \\ P^y_{t+1} &= P^y_t + \delta t\, s_t \sin\Theta_t + W^y_t \\ \Theta_{t+1} &= \Theta_t + \delta t\, \phi_t + W^\theta_t \end{aligned}$$

   where $P_t = [P^x_t \quad P^y_t]^T$ is the robot position relative to a base station, Θ_t is its heading angle, s_t its speed input, ϕ_t its rotation rate input, and w_t is the process noise. Suppose you only get range measurements from two base stations of the form $y_t = \|p_t - p_j\| + v_t$ for station j. Assume that the base station locations are known at $(30m, -5m)$ and $(-10m, 20m)$.

   Let $Q = (0.1\delta t)\text{I}_3$, and $R = 0.1\text{I}_2$, and let the inputs be $s_t = 1m/s$, $\phi_t = (\sin t)rad/s$. Let the initial robot state estimate be normally distributed with mean $\mu_0 = [0m, 0m, 0rad]^T$ and covariance $\Sigma_0 = .01\text{I}_3$. Use $\delta t = 0.5s$ and run for 100 timesteps (i.e., $50s$).

   (a) [8 points] Implement an EKF to estimate the robot state. Check the observability matrix for the linearized C_t, A_t Jacobians at each time step. Is the linearized system observable always, sometimes, never?

Solution:

$$A_t = \begin{bmatrix} 1 & 0 & -\delta t\, s_t \sin(\theta_t) \\ 0 & 1 & \delta t\, s_t \cos(\theta_t) \\ 0 & 0 & 1 \end{bmatrix}$$

$$C_t = \begin{bmatrix} \dfrac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \dfrac{p_t^y - p_{1,y}}{\|p_t - p_1\|} & 0 \\ \dfrac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \dfrac{p_t^y - p_{2,y}}{\|p_t - p_2\|} & 0 \end{bmatrix}$$

The observability matrix $\mathcal{O}$ is given below. We want to check if $\mathcal{O}$ has full column rank at any timestep t (i.e., $\text{rank}(\mathcal{O}) = 3$).

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix}$$

Notice that if we multiply A by itself,

$$A^2 = AA = \begin{bmatrix} 1 & 0 & -\delta t\, s_t \sin(\theta_t) \\ 0 & 1 & \delta t\, s_t \cos(\theta_t) \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -\delta t\, s_t \sin(\theta_t) \\ 0 & 1 & \delta t\, s_t \cos(\theta_t) \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -2\delta t\, s_t \sin(\theta_t) \\ 0 & 1 & 2\delta t\, s_t \cos(\theta_t) \\ 0 & 0 & 1 \end{bmatrix}$$

To simplify notation, notice

$$A^k = \begin{bmatrix} 1 & 0 & -k\,\delta t\, s_t \sin(\theta_t) \\ 0 & 1 & k\,\delta t\, s_t \cos(\theta_t) \\ 0 & 0 & 1 \end{bmatrix}$$

Then,

$$\begin{aligned} CA^k &= \begin{bmatrix} \dfrac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \dfrac{p_t^y - p_{1,y}}{\|p_t - p_1\|} & 0 \\ \dfrac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \dfrac{p_t^y - p_{2,y}}{\|p_t - p_2\|} & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & -k\,\delta t\, s_t \sin(\theta_t) \\ 0 & 1 & k\,\delta t\, s_t \cos(\theta_t) \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \dfrac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \dfrac{p_t^y - p_{1,y}}{\|p_t - p_1\|} & \dfrac{k\,\delta t\, s_t}{\|p_t - p_1\|}\left(-(p_t^x - p_{1,x})\sin(\theta_t) + (p_t^y - p_{1,y})\cos(\theta_t)\right) \\ \dfrac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \dfrac{p_t^y - p_{2,y}}{\|p_t - p_2\|} & \dfrac{k\,\delta t\, s_t}{\|p_t - p_2\|}\left(-(p_t^x - p_{2,x})\sin(\theta_t) + (p_t^y - p_{2,y})\cos(\theta_t)\right) \end{bmatrix} \end{aligned}$$

Putting it all together,

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix} = \begin{bmatrix} \dfrac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \dfrac{p_t^y - p_{1,y}}{\|p_t - p_1\|} & 0 \\ \dfrac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \dfrac{p_t^y - p_{2,y}}{\|p_t - p_2\|} & 0 \\ \dfrac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \dfrac{p_t^y - p_{1,y}}{\|p_t - p_1\|} & \dfrac{\delta t\, s_t}{\|p_t - p_1\|}\left(-(p_t^x - p_{1,x})\sin(\theta_t) + (p_t^y - p_{1,y})\cos(\theta_t)\right) \\ \dfrac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \dfrac{p_t^y - p_{2,y}}{\|p_t - p_2\|} & \dfrac{\delta t\, s_t}{\|p_t - p_2\|}\left(-(p_t^x - p_{2,x})\sin(\theta_t) + (p_t^y - p_{2,y})\cos(\theta_t)\right) \\ \dfrac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \dfrac{p_t^y - p_{1,y}}{\|p_t - p_1\|} & \dfrac{2\,\delta t\, s_t}{\|p_t - p_1\|}\left(-(p_t^x - p_{1,x})\sin(\theta_t) + (p_t^y - p_{1,y})\cos(\theta_t)\right) \\ \dfrac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \dfrac{p_t^y - p_{2,y}}{\|p_t - p_2\|} & \dfrac{2\,\delta t\, s_t}{\|p_t - p_2\|}\left(-(p_t^x - p_{2,x})\sin(\theta_t) + (p_t^y - p_{2,y})\cos(\theta_t)\right) \end{bmatrix}$$

Notice that when $s_t = 0$,

$$\mathcal{O}(s_t = 0) = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix} = \begin{bmatrix} \frac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \frac{p_t^y - p_{1,y}}{\|p_t - p_1\|} & 0 \\ \frac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \frac{p_t^y - p_{2,y}}{\|p_t - p_2\|} & 0 \\ \frac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \frac{p_t^y - p_{1,y}}{\|p_t - p_1\|} & 0 \\ \frac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \frac{p_t^y - p_{2,y}}{\|p_t - p_2\|} & 0 \\ \frac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \frac{p_t^y - p_{1,y}}{\|p_t - p_1\|} & 0 \\ \frac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \frac{p_t^y - p_{2,y}}{\|p_t - p_2\|} & 0 \end{bmatrix}$$

Additionally, what happens when the robot is on the line between the two base stations? Without loss of generality, we move into a coordinate system (i.e., rotate and translate) such that the $p_{1,y} = p_{2,y} = 0$. Then,

$$\mathcal{O}(p_{1,y} = p_{2,y} = 0) = \begin{bmatrix} \frac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \frac{p_t^y}{\|p_t - p_1\|} & 0 \\ \frac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \frac{p_t^y}{\|p_t - p_2\|} & 0 \\ \frac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \frac{p_t^y}{\|p_t - p_1\|} & \frac{\delta t\, s_t}{\|p_t - p_1\|}\left(-(p_t^x - p_{1,x})\sin(\theta_t) + (p_t^y)\cos(\theta_t)\right) \\ \frac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \frac{p_t^y}{\|p_t - p_2\|} & \frac{\delta t\, s_t}{\|p_t - p_2\|}\left(-(p_t^x - p_{2,x})\sin(\theta_t) + (p_t^y)\cos(\theta_t)\right) \\ \frac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & \frac{p_t^y}{\|p_t - p_1\|} & \frac{2\,\delta t\, s_t}{\|p_t - p_1\|}\left(-(p_t^x - p_{1,x})\sin(\theta_t) + (p_t^y)\cos(\theta_t)\right) \\ \frac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & \frac{p_t^y}{\|p_t - p_2\|} & \frac{2\,\delta t\, s_t}{\|p_t - p_2\|}\left(-(p_t^x - p_{2,x})\sin(\theta_t) + (p_t^y)\cos(\theta_t)\right) \end{bmatrix}$$

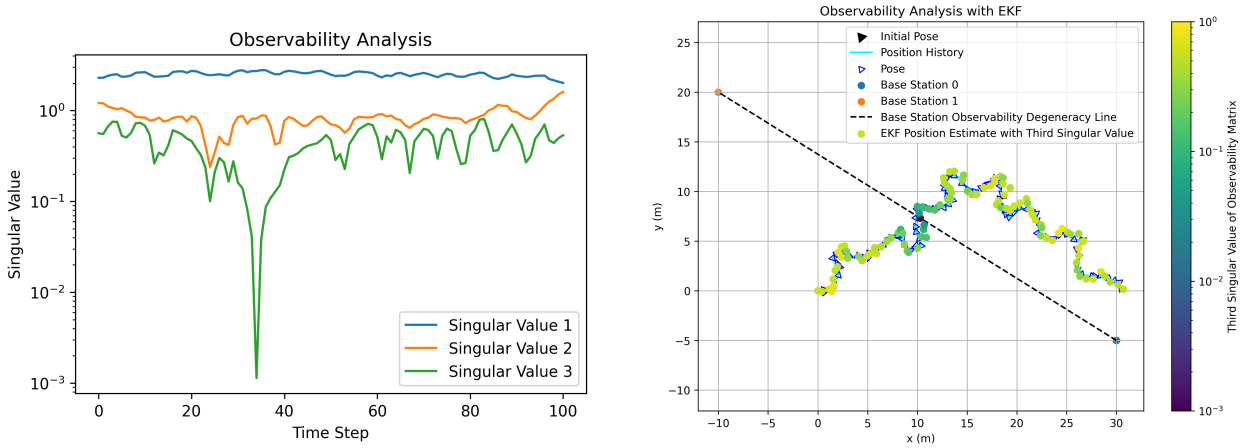
But, notice what happens as $p_t^y \to 0$ (i.e., the robot approaches this line),

$$\mathcal{O}(p_{1,y} = p_{2,y} = p_t^y = 0) = \begin{bmatrix} \frac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & 0 & 0 \\ \frac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & 0 & 0 \\ \frac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & 0 & \frac{\delta t\, s_t}{\|p_t - p_1\|}\left(-(p_t^x - p_{1,x})\sin(\theta_t)\right) \\ \frac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & 0 & \frac{\delta t\, s_t}{\|p_t - p_2\|}\left(-(p_t^x - p_{2,x})\sin(\theta_t)\right) \\ \frac{p_t^x - p_{1,x}}{\|p_t - p_1\|} & 0 & \frac{2\,\delta t\, s_t}{\|p_t - p_1\|}\left(-(p_t^x - p_{1,x})\sin(\theta_t)\right) \\ \frac{p_t^x - p_{2,x}}{\|p_t - p_2\|} & 0 & \frac{2\,\delta t\, s_t}{\|p_t - p_2\|}\left(-(p_t^x - p_{2,x})\sin(\theta_t)\right) \end{bmatrix}$$

So, $\text{rank}(\mathcal{O}) = 3$ if $s_t \neq 0$ and the robot is not colinear with the base stations. Otherwise , $\text{rank}(\mathcal{O}) = 2$ if $s_t = 0$ or if $\exists \alpha \in \mathbb{R}$ such that $p_t = \alpha p_1 + (1-\alpha)p_2$. Lastly, if $\exists \alpha \in \mathbb{R}$ such that $p_t = \alpha p_1 + (1-\alpha)p_2$ (i.e., the robot is colinear with the base station) and *either* $s_t = 0$ or $\theta \to 0$ (in this frame, this means that the robot heading is perpendicular to the line between the base stations), then $\text{rank}(\mathcal{O}) = 1$!

Therefore, the linearized system is observable *if we keep moving without crossing the line between the base stations!* But, it is not *always* observable. Since we are prescribing a constant s_t and we are

unlikely to exactly hit the line between the base stations, the system will *numerically* appear observable throughout the trajectory (Figure 2). However, you should still observe that the filter tends to have larger covariances as it passes between the base stations; specifically, the covariance should expand in the direction perpendicular to the line between the base stations since the update step cannot effectively reduce the covariance in the perpendicular direction (Figure 4).



(a) Singular values over time

(b) Singular values over trajectory

Figure 2: [Plot not required] Singular Values of the Observability Matrix

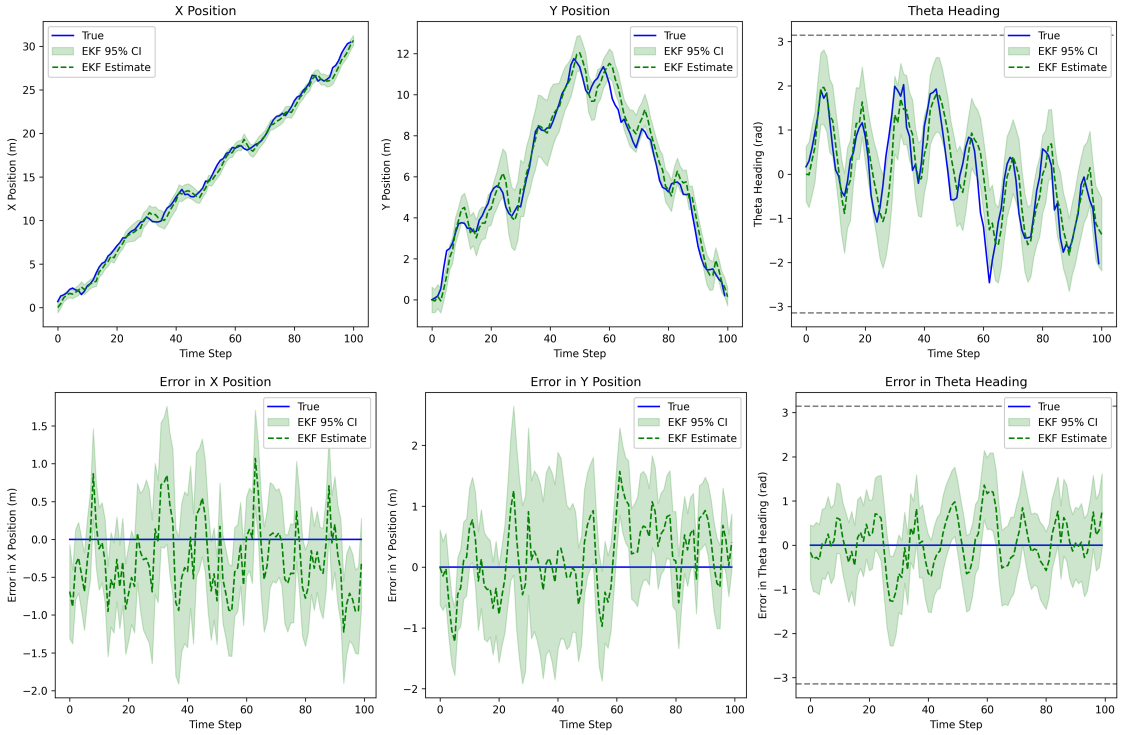


Figure 3: Time History of EKF States and Errors

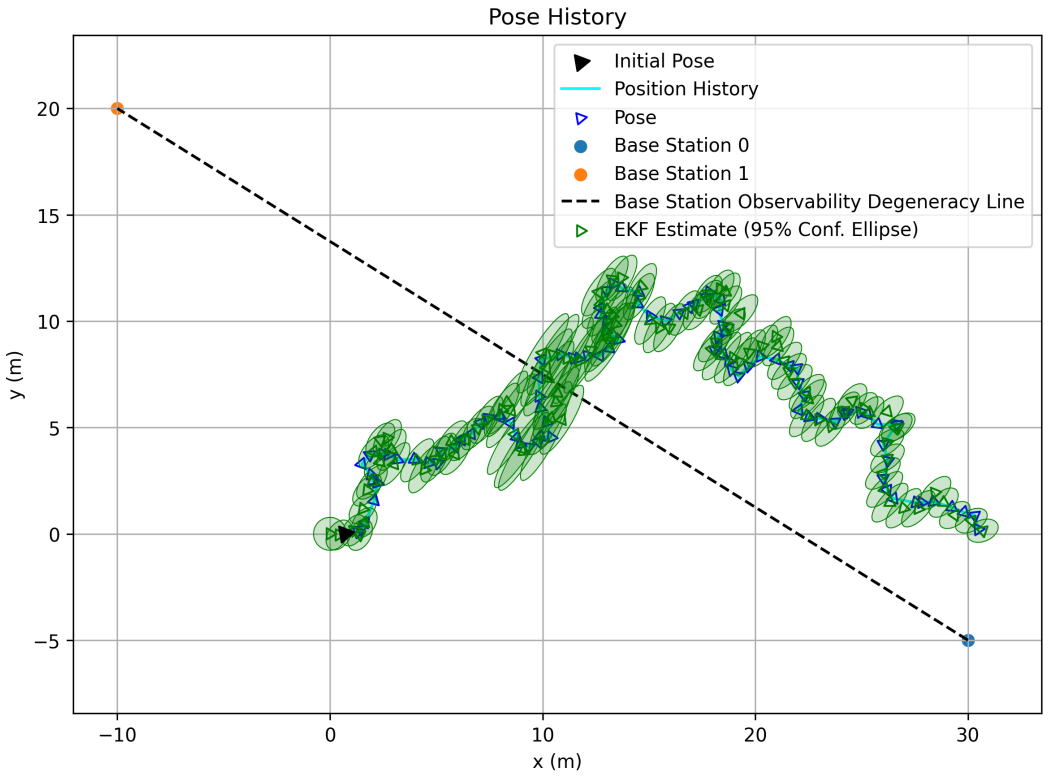


Figure 4: Pose Trajectory of EKF

(b) [8 points] Implement a UKF to estimate the robot state.

Solution: As with the EKF, please refer to Figures 5 and 6.

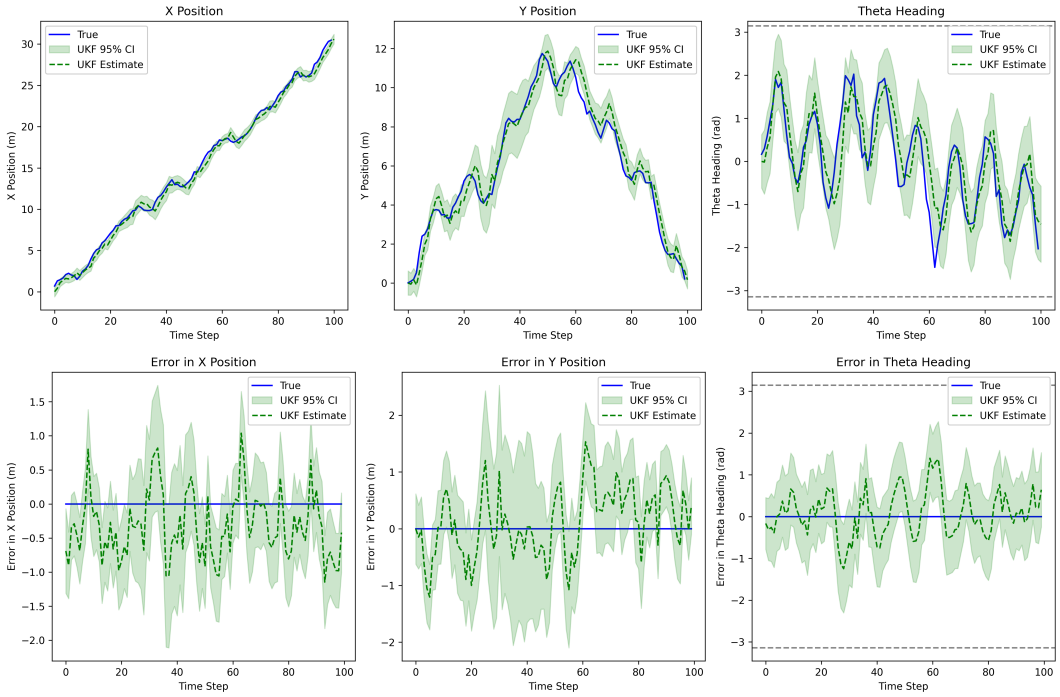


Figure 5: Time History of UKF States and Errors

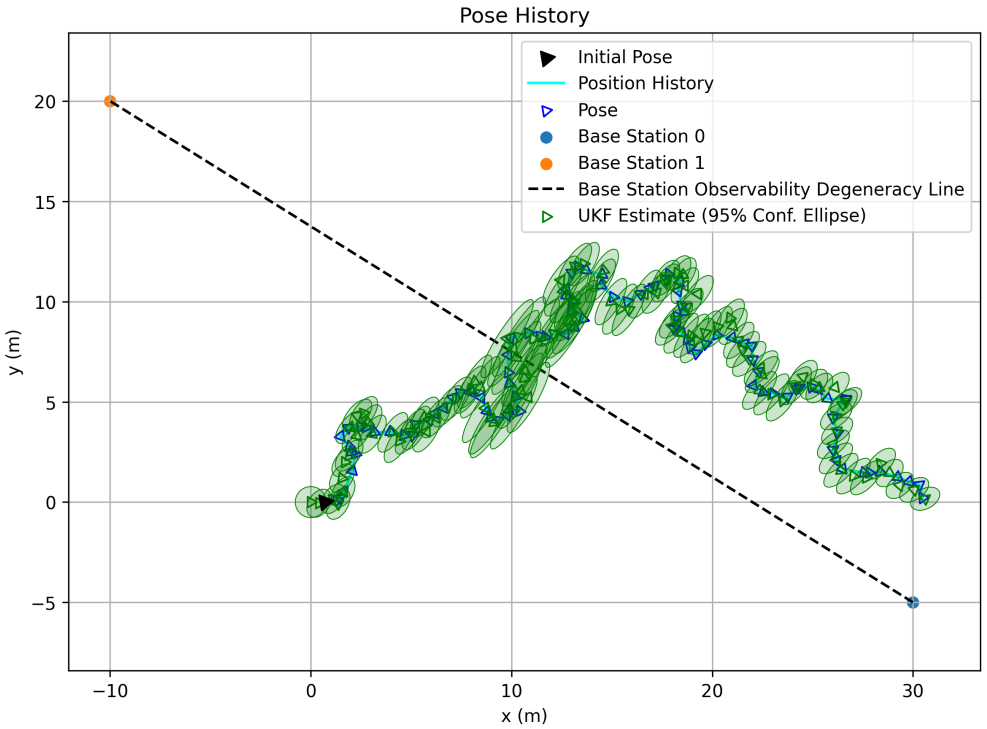


Figure 6: Pose Trajectory of UKF

(c) [8 points] Implement a particle filter with 1000 particles to estimate the robot state.

Solution: There are many ways to visualize the particle filter. You do not need to do all of these. But, since there were questions on Ed about this, here are some options.

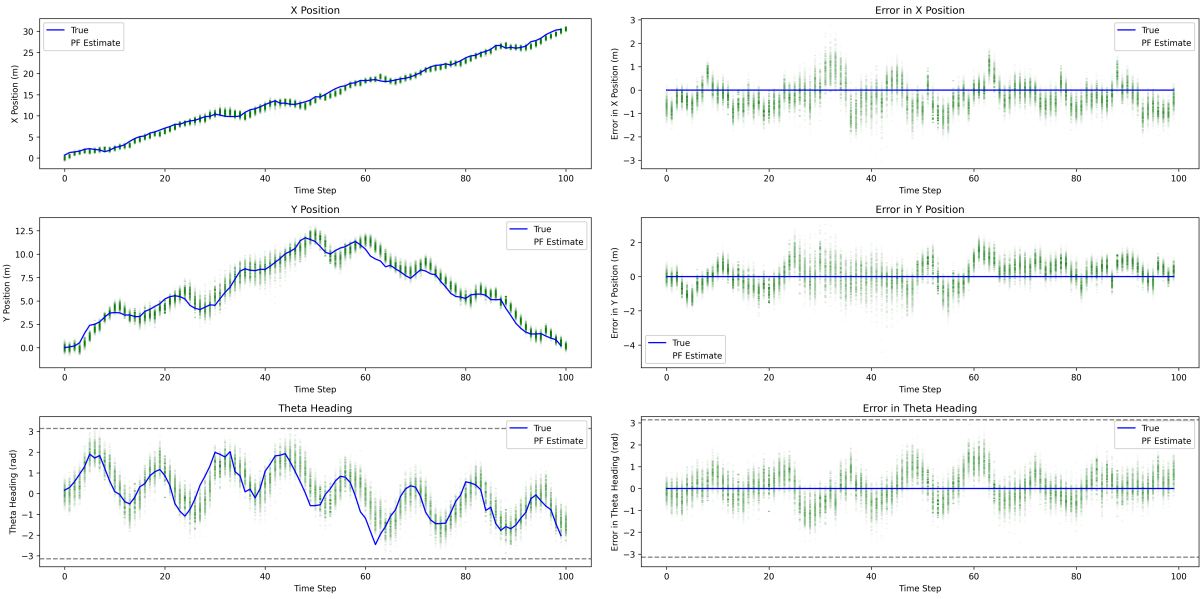


Figure 7: Time History of PF States and Errors as a Scatter Plot

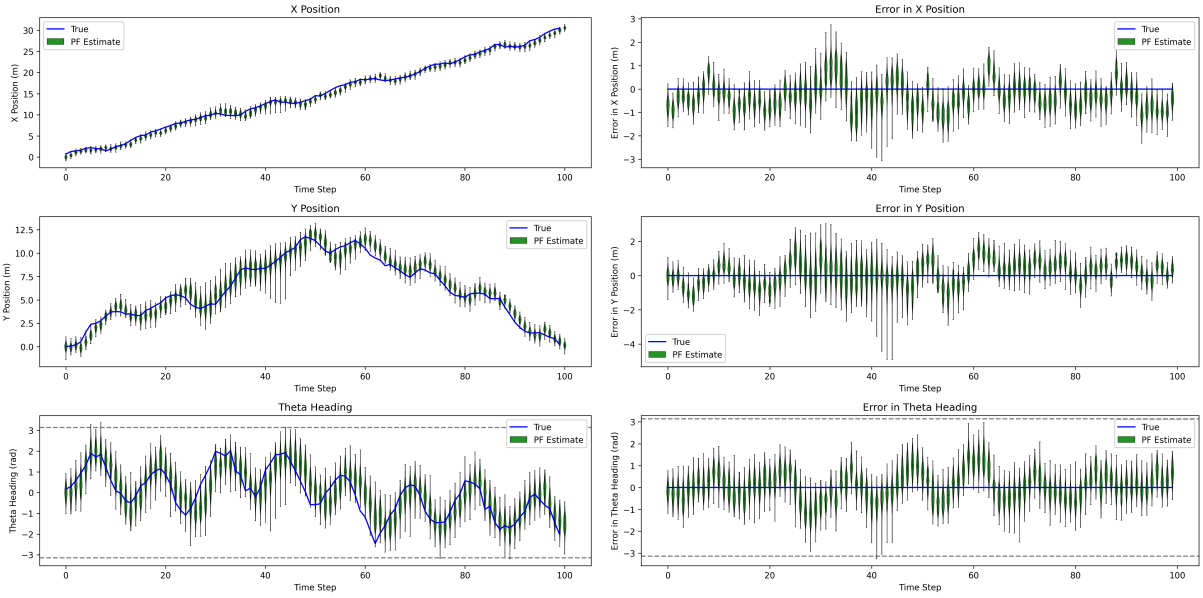


Figure 8: Time History of PF States and Errors as a Violin Plot

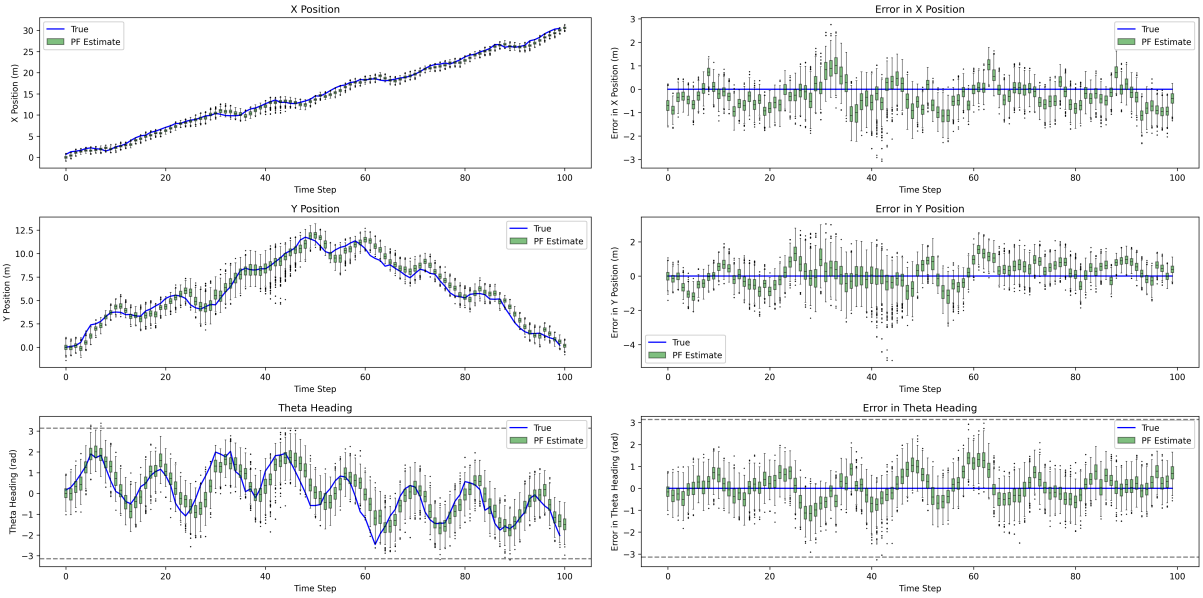


Figure 9: Time History of PF States and Errors as a Box Plot

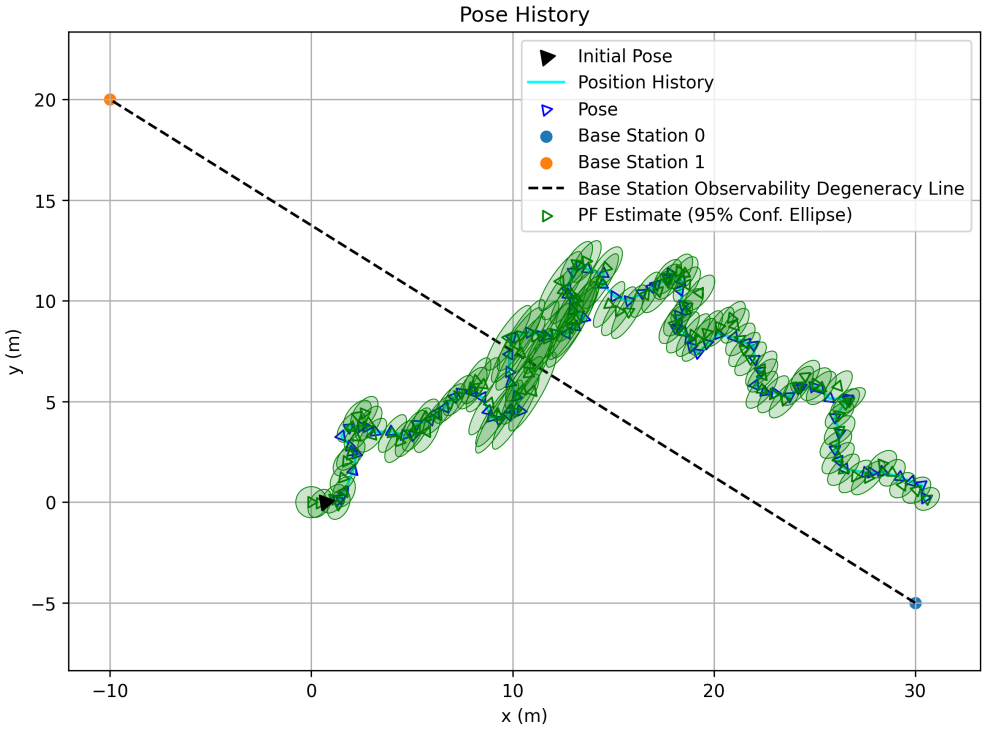


Figure 10: Pose Trajectory of PF with Gaussian Approximation

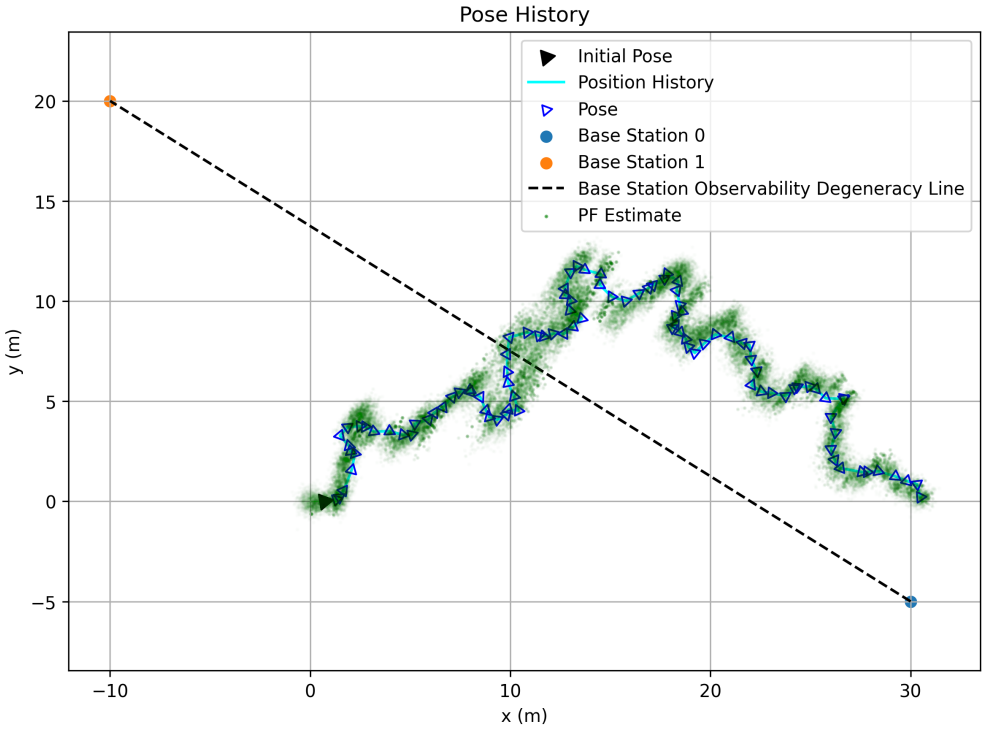


Figure 11: Pose Trajectory of PF with Particles

(d) [3 points] For all the filters, plot the ground-truth trajectories and their estimated trajectories (you may find the provided simulation code useful; see `q3_simulator_script.py`). Use your code from Problem Set 2 to also plot the covariance ellipses for the position component of the state. For the particle filter, be sure to plot the samples and the sample mean. Do the filters track well? Should they be able to (why or why not)?

Solution: For the main required figures of the states, please refer to the previous Figures 3, 4, 5, 6, 7, 8, 9, 10, and 11. The remaining discussion should include remarks about the following:

- All the filters track well, even across seeds, though we only plot one seed in these figures.
- The confidence ellipses do not strictly enclose the full state, but the percentage of time that encloses the full state is reasonable compared to the confidence ellipse probability (i.e., for 95% ellipses and 100 timesteps, roughly 5 timesteps where the ellipse does not enclose the true pose is reasonable).
- Some discussion about how the orientation estimate lags the true orientation since we have to infer the orientation from the dynamics.
- Some discussion that the system is observable, except at sparse, unlucky moments.
- The measurements are technically bimodal, but the correct mode can be recovered through the dynamics. Hence, PF is not strictly necessary since we have a reasonable initialization.
- The PF distributions are quite close to Gaussian (though you do see multimodal distributions at times) since the noise model is Gaussian and the dynamics are close to linear.

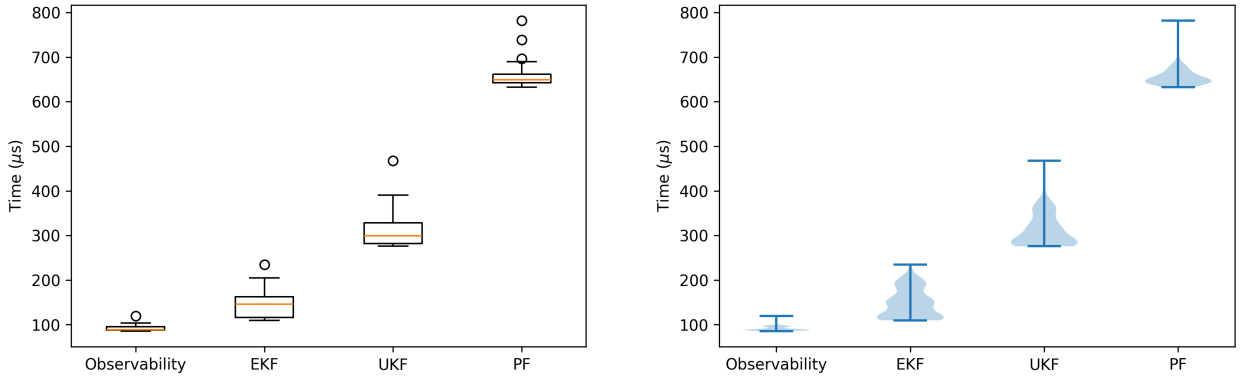
(e) [3 points] Note the average computation time for one loop of each of the three filters. How do they compare?

You can time the filter over the whole trajectory and divide by the trajectory length. However, you may find it more insightful to save the time of each loop into an array and then average the array. The latter approach will let you make a box-plot of the filter timing.

Solution: For the timing analysis, please refer to Figure 12 and 13. The EKF is the fastest and takes roughly half the time of the UKF. This result makes sense since the UKF must pass through multiple

sigma points; however, these computations across sigma points can be vectorized. Students without vectorized code may find that the UKF takes longer to run. Both the EKF and UKF are a fair bit faster than the PF (roughly 4x and 2x, respectively). This result makes sense since many particles in the PF must be considered at run time, which slows down the filter even if vectorized. These filters can be made faster if the `numpy` code is compiled (i.e., with `jax` or `numba`).

Important: If your box plots look collapsed to zero, it is because you are not timing with sufficient significant figures (i.e., you should use `time.perf_counter_ns()` in Python).



(a) Box Plot (b) Violin Plot

Figure 12: [Plot not required] Distribution of time per loop across filters. We add the observability analysis for reference.

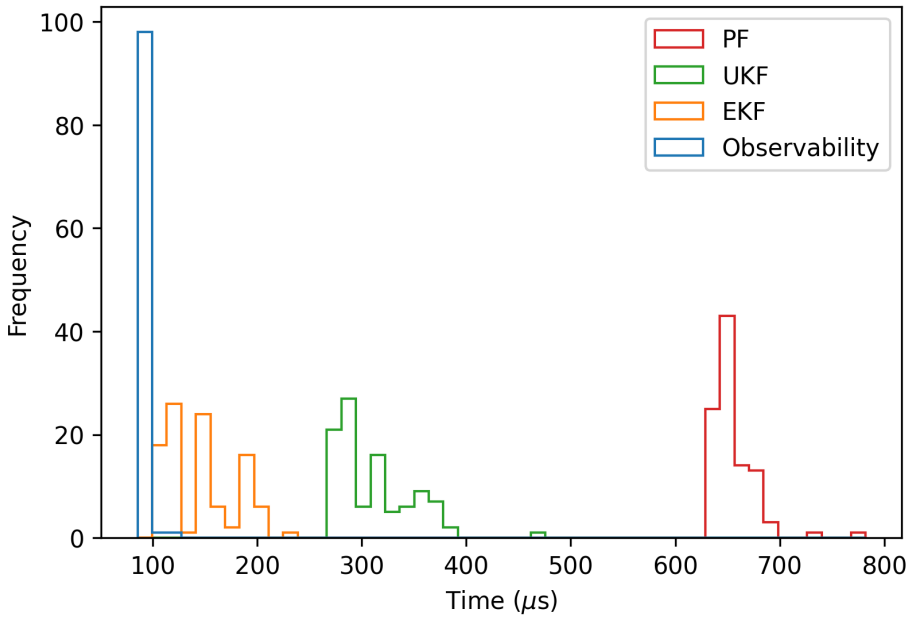


Figure 13: [Plot not required] Distribution of time per loop across filters as a histogram. We add the observability analysis for reference.